DD_ASR-04 - Service Request - Developer Implementation Guide

Version: 1.0 Status: Draft for review Bundle: 10_other Audience: mid-level backend developers, ticketing developers, workflow integration developers, QA Parent DD: DD_ASR-04 - Service Request

1. How To Read This Guide

Service request is a TCK ticket flow with ASR orchestration.

ASR-04 captures the customer request, validates it, optionally gets approval, dispatches it to the owning workflow/module, tracks the result, then resolves the TCK ticket. It does not execute the package change, relocation, equipment swap, or account update itself.

Approvals are not local ASR-04 workflows. If approval is required, ASR-04 calls EM-CFG-04 to evaluate/start the approval process and waits for the approval callback. Camunda is involved only when EM-CFG-04 returns `approvalMode = BPMN`.

2. Step 1 - Seed TCK Service Request Categories

```
insert into ticket_category_config (
  category_id,
  operator_code,
  type_code,
  category_code,
  display_name,
  default_priority,
  default_queue_code,
  default_sla_policy,
  wo_allowed,
  default_wo_kind,
  status
) values
  ('tcat-pkg-upgrade-req', 'WIK', 'GENERAL_INQUIRY', 'PACKAGE_UPGRADE_REQUEST', 'Package upgrade request', 'P3', 'CARE_L1'
  ('tcat-pkg-downgrade-req', 'WIK', 'GENERAL_INQUIRY', 'PACKAGE_DOWNGRADE_REQUEST', 'Package downgrade request', 'P3', 'CA
  ('tcat-relocation-req', 'WIK', 'RELOCATION_SUPPORT', 'RELOCATION_REQUEST', 'Relocation request', 'P2', 'FULFILLMENT_SUPP
  ('tcat-equip-swap-req', 'WIK', 'EQUIPMENT_ISSUE', 'EQUIPMENT_SWAP_REQUEST', 'Equipment swap request', 'P2', 'TECH_SUPPOR
  ('tcat-sip-reset-req', 'WIK', 'TECHNICAL_SUPPORT', 'SIP_PROFILE_RESET', 'SIP profile reset', 'P3', 'TECH_SUPPORT_L1', 'S
  ('tcat-contact-update-req', 'WIK', 'KYC_SUPPORT', 'CONTACT_UPDATE_REQUEST', 'Contact update request', 'P4', 'CARE_L1', '
```

These are TCK categories. ASR-04 adds required fields, dispatch policy, and completion tracking.

3. Step 2 - Seed ASR Service Request Config

```
insert into asr_service_request_type_config (
  config_id,
  operator_code,
  category_code,
  default_type_code,
  owning_module,
  command_endpoint,
  required_fields_json,
  approval_policy_code,
  selfcare_allowed,
  wait_for_external_completion,
  customer_confirmation_required,
  status
) values
  ('asrtc-upgrade', 'WIK', 'PACKAGE_UPGRADE_REQUEST', 'GENERAL_INQUIRY', 'SUB-WF-UPGRADE-01', '/api/subscriptions/{subscri
  ('asrtc-downgrade', 'WIK', 'PACKAGE_DOWNGRADE_REQUEST', 'GENERAL_INQUIRY', 'SUB-WF-DOWNGRADE-01', '/api/subscriptions/{s
  ('asrtc-relocation', 'WIK', 'RELOCATION_REQUEST', 'RELOCATION_SUPPORT', 'SUB-WF-RELOCATION-01', '/api/subscriptions/{sub
  ('asrtc-equip-swap', 'WIK', 'EQUIPMENT_SWAP_REQUEST', 'EQUIPMENT_ISSUE', 'FUL-09', '/api/fulfillment/equipment-swap-requ
  ('asrtc-sip-reset', 'WIK', 'SIP_PROFILE_RESET', 'TECHNICAL_SUPPORT', 'PROV-INT-01', '/api/provisioning/service-profile-r
```

4. Step 3 - Create TCK Ticket

Self-care or Backoffice creates the ticket:

```
POST /api/tickets
Idempotency-Key: sr-ticket-20260605-000014
Content-Type: application/json

{
  "operatorCode": "WIK",
  "typeCode": "GENERAL_INQUIRY",
  "categoryCode": "PACKAGE_UPGRADE_REQUEST",
  "sourceChannel": "SELF_CARE",
  "customerId": "CUS-100045",
  "accountId": "ACC-900045",
  "subscriptionId": "SUB-700045",
  "subject": "Package upgrade request",
  "description": "Customer requests upgrade to Fiber 200M."
}
```

TCK returns:

```
{
  "ticketId": "tck-20260605-000014",
  "statusCode": "OPEN",
  "assignedQueueCode": "CARE_L1"
}
```

5. Step 4 - Create Structured ASR Service Request

The client then posts the structured request:

```
POST /api/asr/service-requests
Content-Type: application/json

{
  "ticketId": "tck-20260605-000014",
  "operatorCode": "WIK",
  "categoryCode": "PACKAGE_UPGRADE_REQUEST",
  "requestPayload": {
    "subscriptionId": "SUB-700045",
    "targetPackageRef": "pkg-fiber-200m",
    "requestedEffectiveDate": "2026-06-06",
    "customerConfirmedPrice": true
  }
}
```

Validate the category config:

```
select *
from asr_service_request_type_config
where operator_code = 'WIK'
  and category_code = 'PACKAGE_UPGRADE_REQUEST'
  and status = 'ACTIVE';
```

Insert:

```
insert into asr_service_request (
  service_request_id,
  ticket_id,
  operator_code,
  category_code,
  request_status,
  request_payload_json,
  validation_result_json,
  created_by_user_id,
  created_at
) values (
  'asr-sr-001',
  'tck-20260605-000014',
  'WIK',
  'PACKAGE_UPGRADE_REQUEST',
  'DRAFT',
  '{"subscriptionId":"SUB-700045","targetPackageRef":"pkg-fiber-200m","requestedEffectiveDate":"2026-06-06","customerConfi
  null,
  'usr-selfcare-100045',
  now()
);
```

6. Step 5 - Validate Request

Call:

```
POST /api/asr/service-requests/asr-sr-001/validate
```

For package upgrade, ASR calls owner-side preview/eligibility APIs:

```
GET /api/subscriptions/SUB-700045
GET /api/packages/pkg-fiber-200m
POST /api/subscriptions/SUB-700045/upgrade-preview
Content-Type: application/json

{
  "targetPackageRef": "pkg-fiber-200m",
  "requestedEffectiveDate": "2026-06-06"
}
```

Expected preview:

```
{
  "eligible": true,
  "currentPackageRef": "pkg-fiber-100m",
  "targetPackageRef": "pkg-fiber-200m",
  "monthlyPrice": 6500.00,
  "currencyCode": "KES",
  "oneOffFees": [],
  "warnings": []
}
```

Patch ASR:

```
update asr_service_request
set request_status = 'VALIDATED',
    validation_result_json = '{"eligible":true,"monthlyPrice":6500.00,"currencyCode":"KES","warnings":[]}'
where service_request_id = 'asr-sr-001';
```

If invalid, set `REJECTED`, add customer-safe reason, and do not dispatch.

If operator policy is complex, run Drools after owner preview. Example outcome:

```
{
  "eligible": true,
  "approvalRequired": false,
  "approvalPolicyCode": null,
  "dispatchAllowed": true
}
```

7. Step 6 - Approval When Required

If config or Drools says approval is required, call EM-CFG-04:

```
POST /api/approvals/evaluate
Content-Type: application/json

{
  "operatorCode": "WIK",
  "moduleCode": "ASR-04",
  "actionCode": "SERVICE_REQUEST_DISPATCH",
  "subjectType": "ASR_SERVICE_REQUEST",
  "context": {
    "categoryCode": "EQUIPMENT_SWAP_REQUEST",
    "sourceChannel": "BACKOFFICE",
    "subscriptionId": "SUB-700045",
    "swapReasonCode": "CUSTOMER_REQUEST",
    "customerConfirmedFee": false
  }
}
```

Possible response:

```
{
  "approvalRequired": true,
  "policyId": "apol-asr-sr-tech-wik-v1",
  "policyCode": "SERVICE_REQUEST_TECHNICAL_APPROVAL",
  "approvalMode": "BPMN",
  "camundaProcessKey": "approval_generic_single_step_v1",
  "firstQueueCode": "TECH_SUPPORT_L2"
}
```

Create the local reference:

```
insert into asr_service_request_approval (
  approval_id,
  service_request_id,
  approval_policy_code,
```

```
  approval_request_id,
  approval_mode,
  camunda_process_instance_id,
  status,
  assigned_queue_code,
  created_at
) values (
  'asra-001',
  'asr-sr-001',
  'SERVICE_REQUEST_TECHNICAL_APPROVAL',
  null,
  'BPMN',
  null,
  'PENDING',
  'TECH_SUPPORT_L2',
  now()
);
```

Start approval request:

```
POST /api/approvals/requests
Idempotency-Key: approval-asr-sr-001
Content-Type: application/json

{
  "operatorCode": "WIK",
  "moduleCode": "ASR-04",
  "actionCode": "SERVICE_REQUEST_DISPATCH",
  "subjectType": "ASR_SERVICE_REQUEST",
  "subjectId": "asr-sr-001",
  "policyId": "apol-asr-sr-tech-wik-v1",
  "callbackUrl": "/internal/asr/service-requests/asr-sr-001/approval-outcome",
  "context": {
    "ticketId": "tck-20260605-000014",
    "categoryCode": "EQUIPMENT_SWAP_REQUEST",
    "requestPayload": {"subscriptionId":"SUB-700045","equipmentInstanceId":"EQ-001"}
  }
}
```

Patch local approval reference with EM-CFG-04 response:

```
update asr_service_request_approval
set approval_request_id = 'apr-sr-001',
    camunda_process_instance_id = 'cam-proc-sr-001'
where approval_id = 'asra-001';
```

Dispatch must return:

```
409 ASR_SR_APPROVAL_REQUIRED
```

until approval is APPROVED.

Approver uses shared EM-CFG-04 task API:

```
POST /api/approvals/tasks/apt-sr-001-step1/approve
Content-Type: application/json

{
  "decisionComment": "Approved equipment swap request."
}
```

EM-CFG-04 calls ASR-04:

```
POST /internal/asr/service-requests/asr-sr-001/approval-outcome
Content-Type: application/json

{
  "approvalRequestId": "apr-sr-001",
  "outcome": "APPROVED",
  "decisionComment": "Approved equipment swap request.",
  "decidedByUserId": "usr-tech-lead-002"
}
```

ASR-04 applies the callback idempotently:

```
update asr_service_request_approval
set status = 'APPROVED',
    actor_user_id = 'usr-tech-lead-002',
    decision_comment = 'Approved equipment swap request.',
    decided_at = now()
where service_request_id = 'asr-sr-001'
  and approval_request_id = 'apr-sr-001'
  and status = 'PENDING';
```

8. Step 7 - Dispatch To Owning Module

For package upgrade:

```
POST /api/asr/service-requests/asr-sr-001/dispatch
Idempotency-Key: asr-sr-001-dispatch-v1
```

ASR records dispatch before the external call:

```
insert into asr_service_request_dispatch (
  dispatch_id,
  service_request_id,
  owning_module,
  command_endpoint,
  idempotency_key,
  request_payload_json,
  dispatch_status,
  retry_count,
  created_at,
  updated_at
) values (
  'asrd-001',
  'asr-sr-001',
  'SUB-WF-UPGRADE-01',
  '/api/subscriptions/SUB-700045/upgrade-requests',
  'asr-sr-001-dispatch-v1',
  '{"source":"ASR-04","sourceTicketId":"tck-20260605-000014","targetPackageRef":"pkg-fiber-200m","requestedEffectiveDate":
  'PENDING',
  0,
  now(),
  now()
);
```

Then call owner:

```
POST /api/subscriptions/SUB-700045/upgrade-requests
Idempotency-Key: asr-sr-001-dispatch-v1
Content-Type: application/json

{
  "source": "ASR-04",
  "sourceTicketId": "tck-20260605-000014",
  "targetPackageRef": "pkg-fiber-200m",
  "requestedEffectiveDate": "2026-06-06",
  "customerConfirmedPrice": true
}
```

Patch dispatch:

```
update asr_service_request_dispatch
set dispatch_status = 'ACKNOWLEDGED',
    owning_ref_id = 'subop-20260605-000019',
    updated_at = now()
where dispatch_id = 'asrd-001';

update asr_service_request
set request_status = 'DISPATCHED'
where service_request_id = 'asr-sr-001';
```

9. Step 8 - Receive Result

Owning module publishes or posts result:

```
POST /api/asr/service-requests/results
Content-Type: application/json

{
  "owningModule": "SUB-WF-UPGRADE-01",
  "owningRefId": "subop-20260605-000019",
  "resultStatus": "COMPLETED",
  "resultCode": "UPGRADE_COMPLETED",
  "customerMessage": "Your package upgrade to Fiber 200M has been completed.",
  "resultPayload": {
    "subscriptionId": "SUB-700045",
    "newPackageRef": "pkg-fiber-200m"
  }
}
```

Insert result:

```
insert into asr_service_request_result (
  result_id,
  service_request_id,
  owning_module,
  owning_ref_id,
  result_status,
  result_code,
  customer_message,
  result_payload_json,
  received_at
) values (
  'asrr-001',
  'asr-sr-001',
  'SUB-WF-UPGRADE-01',
  'subop-20260605-000019',
  'COMPLETED',
  'UPGRADE_COMPLETED',
  'Your package upgrade to Fiber 200M has been completed.',
  '{"subscriptionId":"SUB-700045","newPackageRef":"pkg-fiber-200m"}',
  now()
);

update asr_service_request
set request_status = 'COMPLETED',
    completed_at = now()
where service_request_id = 'asr-sr-001';
```

10. Step 9 - Add Customer Comment And Resolve Ticket

Add customer-visible comment:

```
POST /api/tickets/tck-20260605-000014/comments
Content-Type: application/json

{
  "visibility": "CUSTOMER",
  "commentBody": "Your package upgrade to Fiber 200M has been completed."
}
```

Resolve through TCK:

```
POST /api/tickets/tck-20260605-000014/resolve
Content-Type: application/json

{
  "resolutionCode": "SERVICE_REQUEST_COMPLETED",
  "resolutionSummary": "Package upgrade completed by SUB-WF-UPGRADE-01 operation subop-20260605-000019."
}
```

11. Alternative Flow - Equipment Swap Request

1. Create TCK ticket category `EQUIPMENT_SWAP_REQUEST`.
2. Create ASR service request with `equipmentInstanceId` and `swapReasonCode`.
3. Validate active equipment through OSR instance registry.
4. Dispatch to `FUL-09` or `OSR-RMA-01`.
5. FUL/OSR creates WO when field work is needed.
6. ASR waits for result if configured.
7. Customer sees final swap result, not internal inventory movement details.

12. Alternative Flow - SIP Profile Reset

1. Create TCK ticket category `SIP_PROFILE_RESET`.
2. Validate subscription and service ref.
3. Dispatch to provisioning/profile reset API.
4. Store provisioning command reference.

5. If reset succeeds, resolve ticket.
6. If reset fails due to service fault, convert/link to ASR-01 Technical Trouble.

13. Suggested Worker Commands

These run inside TCK/ASR workers and can be consolidated.

```
asr-sr:dispatch-retry
asr-sr:result-sync
asr-sr:sla-scan
asr-sr:approval-reminder
```

Command	Purpose
`asr-sr:dispatch-retry`	Retries failed dispatch calls using the same idempotency key.
`asr-sr:result-sync`	Polls owning modules for operations that did not emit result events.
`asr-sr:sla-scan`	Finds requests close to SLA breach.
`asr-sr:approval-reminder`	Reminds approval queues about pending approvals.

14. Drools Rule Samples

Use Drools for configurable service-request decisions, not for basic DTO validation.

```
package rules.asr.service_request

rule "Self-care cannot request equipment swap"
when
  $r : ServiceRequestFact(categoryCode == "EQUIPMENT_SWAP_REQUEST", sourceChannel == "SELF_CARE")
  $out : ServiceRequestDecision()
then
  $out.reject("ASR_SR_SELFCARE_NOT_ALLOWED");
end

rule "Equipment swap needs technical approval"
when
  $r : ServiceRequestFact(categoryCode == "EQUIPMENT_SWAP_REQUEST", sourceChannel == "BACKOFFICE")
  $out : ServiceRequestDecision()
then
  $out.setApprovalRequired(true);
  $out.setApprovalPolicyCode("SERVICE_REQUEST_TECHNICAL_APPROVAL");
end

rule "Package upgrade can dispatch after valid preview"
when
  $r : ServiceRequestFact(categoryCode == "PACKAGE_UPGRADE_REQUEST", previewEligible == true, customerConfirmedPrice == tr
  $out : ServiceRequestDecision()
then
  $out.setDispatchAllowed(true);
end
```

15. Error Codes

Code	Meaning
`ASR_SR_CATEGORY_NOT_CONFIGURED`	No active config for category.
`ASR_SR_SELFCARE_NOT_ALLOWED`	Self-care cannot create this request type.
`ASR_SR_REQUIRED_FIELD_MISSING`	Required field is absent.
`ASR_SR_NOT_ELIGIBLE`	Owning module preview says request is not eligible.
`ASR_SR_APPROVAL_REQUIRED`	Approval is required before dispatch.
`ASR_SR_APPROVAL_REQUEST_FAILED`	EM-CFG-04 approval request could not be started.
`ASR_SR_APPROVAL_OUTCOME_MISMATCH`	Approval callback does not match service request/request id.
`ASR_SR_DISPATCH_ALREADY_SENT`	Dispatch already acknowledged.
`ASR_SR_DISPATCH_FAILED`	Owning module command failed.
`ASR_SR_EXTERNAL_OPERATION_OPEN`	Cannot resolve while external operation is open.

Code	Meaning
`ASR_SR_CUSTOMER_MESSAGE_REQUIRED`	Final customer-safe message is missing.

16. Minimum Implementation Order

1. Seed TCK service request categories.
2. Create ASR service request tables and indexes.
3. Seed ASR request type config.
4. Implement guidance API.
5. Implement structured request create API.
6. Implement required-field and self-care validation.
7. Implement owner preview/eligibility adapters.
8. Implement EM-CFG-04 approval evaluate/request/callback integration.
9. Implement dispatch table and owner command adapters.
10. Implement result receive/sync API.
11. Patch TCK resolve validation for service requests.
12. Add Backoffice and self-care screens.
13. Add E2E tests.

17. Test Scenarios

17.1 Self-Care Package Upgrade

- Customer submits package upgrade.
- ASR validates required fields and SUB preview.
- ASR dispatches to SUB workflow.
- SUB returns completed result.
- ASR adds customer comment and resolves TCK ticket.

17.2 Approval Blocks Dispatch

- Category requires supervisor approval.
- User tries dispatch before approval.
- API returns `409 ASR_SR_APPROVAL_REQUIRED`.
- Approval is completed through EM-CFG-04 task API.
- EM-CFG-04 callback marks local approval approved.
- After callback, dispatch succeeds.

17.3 Equipment Swap

- Backoffice creates equipment swap request.
- ASR validates equipment instance.
- ASR dispatches to FUL/OSR.
- FUL/OSR result closes request.

17.4 Downstream Failure

- Owning module rejects request.

- ASR stores failed result.
- Customer-safe reason is added.
- Ticket resolves as rejected or remains open for correction based on policy.

18. Developer Checklist

- Service request uses TCK lifecycle.
- Required fields come from config.
- ASR does not execute owning workflow logic directly.
- Dispatch uses idempotency.
- Approval steps are configured/executed in EM-CFG-04.
- Camunda is used only when EM-CFG-04 policy is BPMN-backed.
- Drools covers configurable eligibility/approval/dispatch decisions where needed.
- Result sync handles both events and polling.
- Ticket closure is blocked until external operation completes when required.
- Customer messages are safe for self-care.